

FINANCIAL AI AGENT

An Intelligent RAG-Based Financial Advisor for Local and Global Markets

Document 3: System Analysis & Design

Graduation Project — Digital Egypt Pioneers Initiative (DEPI)
July 2026

Table of Contents

1. Problem Statement & Objectives

1.1 Problem Statement

Investors operating across the Egyptian and global markets face a fragmented data landscape: global providers frequently lack reliable endpoints for regional currency pairs, and no unified platform combines global equities with local instruments such as Egyptian gold prices. Compounding this, general-purpose conversational AI tools are unsuitable for financial guidance because they hallucinate figures when they lack real-time grounding and are limited by a static training cut-off, leaving them blind to current volatility.

1.2 Objectives

- Provide 100% accurate, real-time price quotes across global and Egyptian markets.
- Ground every AI-generated financial statement in retrieved, verifiable data (RAG) to eliminate hallucination.
- Deliver an Arabic-first, easy-to-use conversational interface that correctly renders mixed RTL/LTR content.
- Extend the platform beyond reporting into forecasting via a validated, risk-audited predictive ML pipeline.

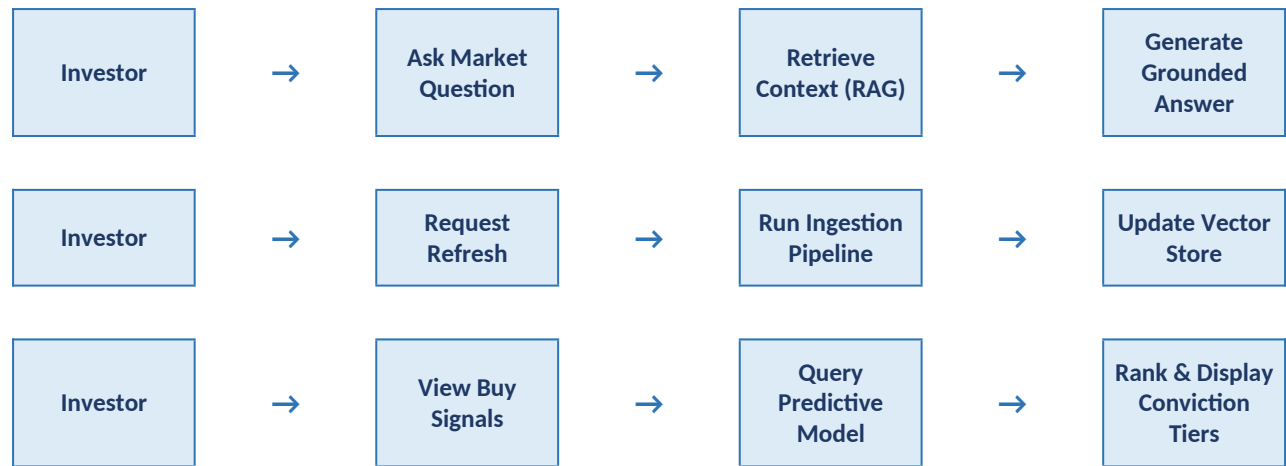
2. Use Case Diagram & Descriptions

2.1 Actors

- Investor (Egyptian / Global) — the primary end user asking financial questions.
- Data Ingestion Scheduler — automated process triggering scheduled or on-demand data refresh.
- Predictive Model Pipeline — the offline-trained ML system supplying buy-signal probabilities.

2.2 Use Case Overview (textual diagram)

The diagram below represents the system boundary with actors on the outside and use cases as ovals inside; it is rendered here as a structured flow for document portability.



2.3 Use Case Descriptions

Use Case	Actor	Description
Ask Market Question	Investor	User submits a natural-language question about a price, currency, or trend; system retrieves grounded context and returns an answer
Request Data Refresh	Investor / Scheduler	Triggers re-ingestion of global and local data sources before answering a critical query
View Buy Signals	Investor	User requests the current list of high/moderate-conviction stocks per the predictive model
Refuse Out-of-Scope Query	System (internal)	When a question is unrelated to finance, the LLM boundary-setting rule causes an explicit refusal

3. Functional & Non-Functional Requirements (Summary)

Full requirement listings are maintained in Document 2 (Literature Review & Requirements Gathering). Key capabilities and constraints are summarized here for design traceability.

3.1 Functional Capabilities

- Dual-market real-time data ingestion (global via yfinance; local via BeautifulSoup scraping of Isagha).
- RAG-grounded conversational answers via Llama-3.3-70b-versatile (Groq).
- Arabic-first UI with RTL/LTR-safe numeric formatting.
- Predictive classification of statistically significant 10-day price moves, ensembled across XGBoost seeds.
- Risk-adjusted backtesting and signal ranking (high/moderate conviction).

3.2 Non-Functional Constraints

- Near-instantaneous chat response latency (Groq LPU inference).
- 100% uptime for regional currency pairs via mathematical cross-rate fallback.
- Local, privacy-preserving embeddings (no third-party embedding API).
- No data leakage: embargoed time-series cross-validation (20-day gap).
- Model changes accepted only when improvement exceeds a 0.02 AUC noise-tolerance threshold.

4. Software Architecture

The system follows a layered, service-oriented architecture separating data ingestion, knowledge storage/retrieval, the conversational AI layer, the predictive ML layer, and the presentation layer.



4.1 Component Responsibilities

Component	Responsibility
Data Sources	yfinance API (global equities/FX) and a custom BeautifulSoup scraper (Isagha, Egyptian gold)
Ingestion Layer	Fetches, cleans, validates (OHLC consistency, deduplication), and normalizes incoming data on schedule or on-demand refresh
Embedding Model	Local HuggingFace BAAI/bge-m3 deployment converts ingested text/data into vector representations
Storage & Retrieval	PostgreSQL extended with pgvector acts as both the relational store and the semantic vector search engine
LLM Engine	Groq-hosted Llama-3.3-70b-versatile generates the conversational response, constrained to retrieved context via role- and boundary-prompting
Predictive ML Pipeline	Offline-trained XGBoost ensemble (financial_advisor.ipynb) scores tickers for statistically significant 10-day moves; results surfaced to the chat/reporting layer
Frontend	Streamlit web application providing the conversational interface, live-refresh control, and session state

4.2 Architectural Style

The design follows a pipeline / layered architecture rather than strict MVC or microservices: each layer (ingestion → storage/retrieval → generation → presentation) is independently replaceable, which supports the planned extensions (EGX, crypto, alerting) without redesigning upstream components.

5. Database Design & Data Modeling

5.1 Entity-Relationship Diagram (textual)

The core schema centers on market instruments and their time-series prices, the vectorized knowledge base used for retrieval, and the model output tables used for conviction ranking.

Entity	Key Attributes	Relationships
Instrument	instrument_id (PK), ticker, sector_id (FK), market_type (global/local)	1—N with PriceObservation; N—1 with Sector
Sector	sector_id (PK), sector_name	1—N with Instrument
PriceObservation	obs_id (PK), instrument_id (FK), date, open, high, low, close, volume	N—1 with Instrument
KnowledgeChunk	chunk_id (PK), source, content_text, embedding (vector), created_at	N—1 with Instrument (nullable, for market-linked chunks)
ModelPrediction	prediction_id (PK), instrument_id (FK), as_of_date, buy_probability, conviction_tier	N—1 with Instrument
ChatSession	session_id (PK), user_ref, started_at	1—N with ChatMessage
ChatMessage	message_id (PK), session_id (FK), role, content, created_at	N—1 with ChatSession

5.2 Logical & Physical Schema Notes

- PriceObservation is unique on (instrument_id, date) and indexed on date for range scans, mirroring the drop_duplicates(['Ticker', 'Date']) rule applied in the ingestion notebook.
- KnowledgeChunk.embedding uses the pgvector `vector` type sized to the BAAI/bge-m3 output dimension, with an IVFFlat/HNSW index for approximate nearest-neighbor search.
- ModelPrediction.conviction_tier is derived (high / moderate) from the calibrated probability thresholds produced during model validation, not stored redundantly at write time beyond a cached label for query performance.
- Sector is a lookup table mirroring the SECTOR_MAP grouping (Technology, Consumer, Financials, Healthcare, Staples, Industrials, Energy) used for sector-relative feature engineering.
- Normalization: the schema is in 3NF; ModelPrediction and PriceObservation avoid storing derived indicators (RSI, MACD, etc.) directly — those remain transient features computed by the ML pipeline rather than persisted columns, keeping the physical schema lean.

6. Data Flow & System Behavior

6.1 Context-Level Data Flow Diagram (DFD Level 0)

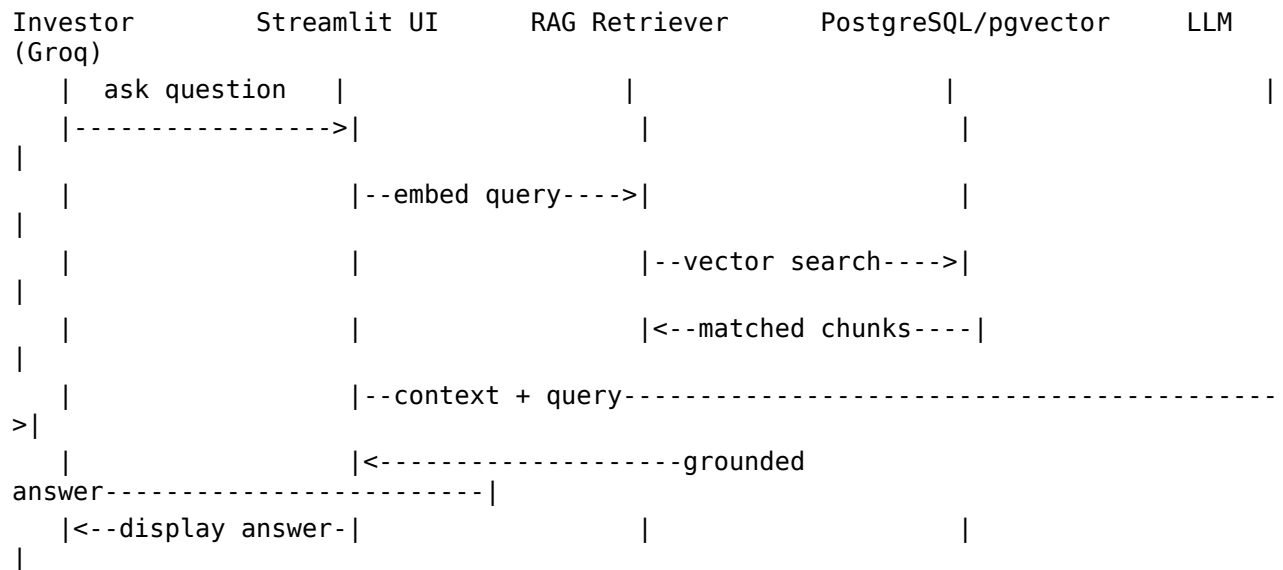


The system receives questions and refresh requests from the Investor and pulls market data from external Data Sources, returning grounded answers and buy-signal rankings to the Investor.

6.2 Level-1 Data Flow Diagram



6.3 Sequence Diagram — "Ask Market Question" (textual)



6.4 Activity Diagram — Data Ingestion & Buy-Signal Generation (textual)

- Start → Fetch global prices (yfinance) and scrape local gold price (Isagha)
- Run data-quality checks (OHLC bounds, volume ≥ 0 , deduplicate on ticker+date)
- Compute engineered features (normalized OHLC, moving averages, RSI, MACD, Bollinger Bands, ATR, OBV, momentum, sector-relative and cross-sectional rank features)
- Branch: if training mode → embargoed cross-validation → hyperparameter search → ensemble fit
- Branch: if inference mode → load persisted ensemble → score latest observation per ticker
- Rank tickers by probability against calibrated thresholds → assign conviction tier
- End → Store predictions / surface to chat and reporting layer

7. State Diagram

The diagram below models the lifecycle of a Chat Session object as it moves through interaction states.



- Idle → Awaiting Query: triggered when a session starts or the previous answer is delivered.
- Awaiting Query → Retrieving Context: triggered by the user submitting a question.
- Retrieving Context → Generating Response: triggered once relevant chunks are returned from pgvector.
- Generating Response → Response Delivered: triggered when the LLM completes generation (or issues an explicit refusal for out-of-scope input).
- Response Delivered → Awaiting Query: session persists state and loops for follow-up questions.

8. Class Diagram

Key classes and their principal attributes/methods are summarized below (rendered as a structured table for portability).

Class	Key Attributes	Key Methods
MarketDataIngestor	tickers: list, source: enum	fetch_global(), fetch_local(), clean(), validate()
Embedder	model_name: str = 'BAAI/bge-m3'	embed(text): vector
VectorStore	connection, table_name	upsert(chunk), search(query_vec, k): List[Chunk]
RAGRetriever	store: VectorStore, embedder: Embedder	retrieve(question): List[Chunk]
FinancialLLMAgent	model: 'llama-3.3-70b-versatile', system_prompt	answer(question, context): str, refuse_if_out_of_scope()
PredictiveModel	ensemble: List[XGBClassifier], features: List[str], thresholds: dict	predict_proba(df), rank_conviction(df)
ChatSession	session_id, history: List[ChatMessage]	add_message(), get_context()
BacktestEngine	test_df, cost_bps, hold_days	run(probs, threshold): BacktestResult

Relationships: FinancialLLMAgent uses RAGRetriever and VectorStore (composition); RAGRetriever uses Embedder; ChatSession aggregates ChatMessage; PredictiveModel is used by the reporting layer to populate ModelPrediction records; BacktestEngine consumes PredictiveModel output for evaluation.

9. UI/UX Design & Prototyping

9.1 Wireframes & Mockups (description)

- Main chat view: central conversational thread (ChatGPT-style bubbles), input box pinned to the bottom, and a persistent 'Update Data' button in the header.
- Sidebar / panel: current conviction-tier ticker list (high vs. moderate), refreshed after each ingestion run.
- Response formatting: numeric values and tickers are wrapped in brackets (e.g., [EGP 4,150] or [AAPL]) so they render as isolated left-to-right runs inside right-to-left Arabic paragraphs.

9.2 UI/UX Guidelines

- Language: Arabic-first labels and prompts, with English fallback for tickers and technical terms.
- Typography: a typeface with full Arabic glyph support and clear numeral rendering.
- Color & accessibility: high-contrast text for numeric data; consistent color coding for high- vs. moderate-conviction signals.
- Bidirectional text handling: enforced at the prompt-engineering level (bracketing) rather than left to client-side CSS alone, ensuring consistent rendering regardless of the rendering surface.

10. System Deployment & Integration

10.1 Technology Stack

Layer	Technology
Data ingestion	Python, yfinance, BeautifulSoup
Storage & retrieval	PostgreSQL + pgvector extension
Embeddings	HuggingFace BAAI/bge-m3 (local deployment)
LLM inference	Groq API — Llama-3.3-70b-versatile
Predictive ML	XGBoost, LightGBM (optional ensemble member), scikit-learn
Frontend	Streamlit

10.2 Deployment Diagram (textual)



The Streamlit application server hosts the frontend and orchestrates calls to the local PostgreSQL/pgvector instance and the external Groq inference API; the predictive ML pipeline runs as a scheduled offline job whose outputs are written back to PostgreSQL.

10.3 Component Diagram (textual)



Each service is a loosely-coupled component communicating through the shared PostgreSQL store, allowing independent deployment/scaling and supporting the planned extensions (EGX, crypto, alerting, portfolio optimization).

11. Additional Deliverables

11.1 API Documentation (if applicable)

Internal service boundaries (ingestion, retrieval, scoring) are currently invoked in-process from the Streamlit app rather than exposed as public APIs. Should external API access be added, endpoints would follow REST conventions: GET /prices/{ticker}, GET /predictions/{ticker}, POST /chat.

11.2 Testing & Validation

- Unit tests: data-quality filters (OHLC bounds, deduplication), feature-engineering functions, and threshold-selection logic.
- Integration tests: end-to-end ingestion → embedding → retrieval → LLM response path; ingestion → feature build → prediction path.
- Model validation: embargoed cross-validation (20-day gap) across 5 folds; ensemble stability check via repeated re-training with different random seeds (observed test AUC non-determinism range: 0.0000–0.0014).
- User acceptance testing: sample question sets in both Arabic and English, verifying grounded answers, refusal behavior for out-of-scope queries, and correct RTL/LTR rendering.

11.3 Deployment Strategy

- Hosting: containerized Streamlit application with a managed PostgreSQL instance (pgvector-enabled).
- Pipelines: scheduled ingestion job (cron or equivalent) plus an on-demand refresh triggered from the UI.
- Scaling: stateless frontend instances behind a load balancer; the vector store and offline ML pipeline scale independently of the chat layer.
- Model refresh: the predictive ensemble is retrained periodically offline and the serialized ensemble/thresholds are reloaded by the scoring service without redeploying the frontend.